

รายงานการออกแบบและตัวอย่างโค้ดสำหรับ 3D Surface Map ของ MotorBoy Dashboard

สรุปผู้บริหาร

สำหรับงาน **fuel / ignition remap editor** ที่ต้อง “หน้าต่างเหมือนภาพอ้างอิงให้มากที่สุด” และฝังในแดชบอร์ด HTML ของ MotorBoy ได้จริง ผมแนะนำให้ใช้ **three.js** เป็นแกนหลัก แล้วเสริมด้วย **OrbitControls** สำหรับหมุนกล้อง และ **EffectComposer + RenderPass + UnrealBloomPass** สำหรับเอฟเฟกต์นีออน/บลูม เพราะชุดนี้ให้การควบคุม geometry, wireframe, กล้อง, post-processing และการจัดวางใน panel ได้ละเอียดที่สุดในฝั่ง browser/WebGL. เอกสารทางการของ three.js ระบุว่า OrbitControls รองรับ orbit/dolly/pan, EffectComposer ใช้จัดลำดับ post-processing passes, และ UnrealBloomPass เน้นคุณภาพพร้อมความคุ้มค่าด้าน performance โดยต้องเปิด tone mapping ที่ renderer. ¹

ถ้าต้องการทำ prototype ให้เร็วมาก ๆ แบบกราฟวิเคราะห์ข้อมูล **Plotly.js** เป็นตัวเลือกที่ดี เพราะมี **surface trace, contour, lighting** และ **wireframe-like mode** จาก **contours / hidesurface** อยู่แล้ว แต่ความยืดหยุ่นเชิง visual-polish เพื่อให้ได้ cyberpunk neon แบบภาพอ้างอิงจะน้อยกว่า three.js และ bundle เต็มค่อนข้างใหญ่กว่า. ²

Babylon.js ก็ทำงานนี้ได้ดีเช่นกัน โดยเฉพาะอย่างยิ่งถ้าทีมชอบ abstraction ของ engine และอยากใช้กล้องแบบ orbit กับ glow ของ emissive mesh ผ่าน **ArcRotateCamera** และ **GlowLayer**; แต่ถ้าวัตถุประสงค์คือ “มี surface map editor 1 panel ในแดชบอร์ด” three.js จะเบาและตรงโจทย์กว่า. ส่วน **deck.gl** เหมาะกับงาน large-scale layered data visualization มากกว่า และ **regl** ให้ความยืดหยุ่นสูงสุดแต่จะกลายเป็นงาน shader-engineering มากขึ้น จึงไม่ใช่ตัวเลือกแรกสำหรับ MotorBoy dashboard รอบนี้. ³

ไฟล์ตัวอย่าง single-file ที่รันได้จริงถูกสร้างไว้แล้วที่: [ดาวน์โหลดไฟล์ HTML ตัวอย่าง](#)

สถาปัตยกรรมและโลจิสติกส์ที่แนะนำ

สแต็กที่แนะนำ

แกนหลักที่แนะนำคือ:

1. **three.js** สำหรับสร้าง surface mesh, back-grid, wireframe lattice, กล้อง และการ resize ตาม panel
2. **OrbitControls** สำหรับการหมุน/ซูมกล้องในลักษณะเดียวกับกราฟ 3D ในภาพอ้างอิง
3. **EffectComposer + RenderPass + UnrealBloomPass** สำหรับ neon glow / bloom แบบ cyber-tech
4. **Canvas 2D fallback** สำหรับ accessibility และกรณี WebGL ไม่พร้อม
5. **JSON import/export** เป็น format กลางของ map เพื่อใช้กับ backend หรือแอปจริงในอนาคต

เหตุผลที่เลือกชุดนี้คือ three.js ใช้ **PlaneGeometry** และ **BufferGeometry** เป็นฐานได้ตรงไปตรงมา, addon imports ใช้รูปแบบเดียวกับเอกสารทางการ (**three/addons/...**), OrbitControls ต้องเรียก **update()** เมื่อเปิด damping, และ UnrealBloomPass เป็น bloom pass ที่เอกสารทางการระบุว่าคุณภาพดีและคุ้มค่าด้าน performance เมื่อเปิด tone mapping ที่ renderer. ⁴

แผนเทคนิคแบบย่อ

แนวทาง implementation ที่แนะนำสำหรับเวอร์ชัน production-ready คือ:

1. เริ่มจาก **control grid** ขนาดเล็ก แบบที่แก้ไขจริงได้ เช่น `9 × 8`
2. ทำ **bilinear interpolation** ไปเป็น render grid ที่ละเอียดขึ้น เช่น `64 × 48`
3. สร้าง `PlaneGeometry(width, depth, cols-1, rows-1)` แล้ว map ค่าไปเป็นความสูงแกน Y
4. ลงสีด้วย **thermal/neon gradient** ต่อจุด (vertex color)
5. วาด **wireframe overlay** แยกจาก mesh surface เพื่อคุมสไตล์ได้ดีกว่า wireframe สามเหลี่ยมอัตโนมัติ
6. ใส่ **camera preset** สำหรับ 3D และ 2D view
7. ใช้ `ResizeObserver` เพื่อให้ panel ขยาย/ย่อได้โดยไม่เกิด page scroll
8. ทำ `JSON export/import` เป็น interface ระหว่าง UI กับระบบจริง

```
timeline
  title ลำดับงานพัฒนา 3D Surface Remap Editor
  requirement : panel size, no-scroll, neon theme, input model
  MVP WebGL : scene, camera, orbit, dense surface mesh
  Visual polish : thermal gradient, lattice wireframe, cage grid, bloom
  UX controls : 2D/3D toggle, zoom +/-, preset selector, export/import
  Dashboard integration : ResizeObserver, fixed panel fit, 2D fallback
  Production hardening : perf tuning, JSON schema, QA, handoff
```

ทางเลือกที่ไม่ใช่เป็นตัวหลัก

Plotly.js เหมาะมากถ้าเป้าหมายคือกราฟ 3D สำเร็จรูปเร็วที่สุด เพราะ `surface` trace มี contour, lighting, hover, และสามารถทำ wireframe-style plot ได้จาก `contours.*` ร่วมกับ `hidesurface`. แต่การทำ neon cyberpunk แบบ reference image ที่ต้องคุมกล้อง, glow, scene cage, overlay labels แบบเฉพาะทาง จะถูกจำกัดด้วย abstraction เชิง chart มากกว่า scene graph ของ three.js. ⁵

Babylon.js รองรับ orbit camera ผ่าน `ArcRotateCamera` และ glow จาก emissive mesh ผ่าน `GlowLayer` ได้ดีมาก จึงเป็นตัวเลือกที่ดีถ้าโปรเจกต์มี 3D หลายส่วนและอยากใช้ engine เดิมรูปแบบ. แต่สำหรับ dashboard panel เดียว three.js จะตรงโจทย์กว่าในเชิง footprint และ flexibility. ⁶

deck.gl จากเอกสารทางการวางตัวเป็น GPU-powered framework สำหรับ large-scale exploratory visualization แบบ layer-driven; จาก layer catalog ที่เปิดเผย ไม่มี primitive surface-map editor สำเร็จรูปแบบที่สวมเข้ากับ remap panel นี้ได้โดยตรง จึงต้องเขียน custom layer เองพอสมควร. **regl** ในทางกลับกันก็ให้ control ต่ำระดับ canvas/GLSL มาก ซึ่งดีมากสำหรับทีมกราฟิก แต่จะเพิ่มต้นทุนวิศวกรรมเกินความจำเป็นสำหรับ MotorBoy use-case รอบนี้. ⁷

เปรียบเทียบ three.js, Plotly.js และ Babylon.js

ตารางนี้สรุป “ความเหมาะสมกับงาน remap editor ในแดชบอร์ด” มากกว่าการตัดสินว่าไลบรารีไหนดีกว่าในทุกงาน

ไลบรารี	จุดแข็งเด่น	ข้อจำกัดใน use-case นี้	ขนาด payload โดยประมาณ*	ความเหมาะสมกับ MotorBoy panel
three.js	คุม scene, mesh, wireframe, camera, bloom, overlay, resize ได้ละเอียดมาก	ต้องเขียน scene/ render loop เองมากกว่า Plotly	ตัวอย่าง CDN snapshot: <code>three.module.js</code> ~570 kB และ <code>three.module.min.js</code> ~337 kB ในเวอร์ชันหนึ่งของ แพ็กเกจ	แนะนำที่สุด
Plotly.js	surface chart สำเร็จรูปเร็ว, contour/lighting/ hover พร้อมใช้	ปรับ visual ให้เหมือน cyberpunk scene ยากกว่า, full bundle ใหญ่	ตัวอย่าง <code>plotly.min.js</code> full dist ~4.84 MB	เหมาะสมเป็น prototype / analytics screen
Babylon.js	ArcRotateCamera, emissive glow, engine abstraction ดี	ถ้าใช้แค่ panel เดียว จะดูเป็น engine ใหญ่ เกินงาน	ตัวอย่าง <code>babylon.js</code> UMD ~8.22 MB	เหมาะถ้าโปรเจกต์มี 3D หลายส่วน

*ตัวเลขเป็น ตัวอย่างจาก **CDN/package snapshot** และขึ้นกับเวอร์ชัน, การบีบอัด, และรูปแบบ bundle ที่ใช้งานจริง. เอกสารทางการ/ตัวอย่างที่ใช้เปรียบเทียบอยู่ที่ , , ; ส่วน snapshot ขนาด package อ้างอิงจาก CDN ของ three, Plotly และ Babylon ตามผลค้นหาที่เปิดได้. ⁸

ข้อสรุปเชิงปฏิบัติคือ ถ้าต้อง “เหมือนภาพอ้างอิงที่สุด” และต่อเข้ากับ dashboard HTML ได้ดีที่สุด ให้ใช้ **three.js** เป็น **production path**; ถ้าต้อง “ได้กราฟเร็วที่สุดก่อน” ให้ใช้ Plotly.js เป็น prototype path; ถ้าระยะยาว platform จะมี 3D จำนวนมากทั้งแอป ค่อยพิจารณา Babylon.js เป็น platform choice. ⁹

ไฟล์ตัวอย่าง single-file ที่รันได้จริง

โค้ด single-file HTML/CSS/JS ฉบับเต็มที่รันได้ทันทีและมีคอมเมนต์อธิบาย key parts อยู่ในไฟล์นี้:

[ดาวน์โหลดไฟล์ HTML ตัวอย่าง](#)

ไฟล์ตัวอย่างนี้ทำครบตามที่ขอ:

- รันด้วย WebGL ผ่าน **three.js**
- ใช้ **neon purple → magenta → red thermal gradient** พร้อม lowlands อม blue ให้ใกล้เคียง reference
- มี **wireframe lattice overlay**
- มี **axis labels** และ **right-scale 0-100**
- มี **OrbitControls**
- มี **2D / 3D toggle**
- มี **zoom +/-**
- fit กับ **fixed dashboard panel**
- ไม่มี **page scroll**

- มี **Canvas 2D fallback**
- ใช้ **vector geometry** ทั้งหมด ไม่ต้องพึ่ง PNG/SVG ภายนอก

CDN ที่ใช้ในตัวอย่าง

โค้ดใช้ import map และอ้างอิง CDN แบบเจาะเวอร์ชันตรง ๆ ดังนี้

```
<script type="importmap">
{
  "imports":{
    "three": "https://unpkg.com/three@0.184.0/build/three.module.js",
    "three/addons/": "https://unpkg.com/three@0.184.0/examples/jsm/"
  }
}
</script>
```

จากนั้น import แบบเดียวกับเอกสารทางการ:

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import { EffectComposer } from 'three/addons/postprocessing/EffectComposer.js';
import { RenderPass } from 'three/addons/postprocessing/RenderPass.js';
import { UnrealBloomPass } from 'three/addons/postprocessing/UnrealBloomPass.js';
```

รูปแบบ import นี้สอดคล้องกับ docs ของ OrbitControls, EffectComposer และ RenderPass โดยตรง. 10

วิธีฝังเข้าไปใน MotorBoy dashboard เดิม

เวลาฝังเข้ากับ dashboard จริง ให้เปลี่ยนจาก full-screen wrapper ไปเป็น **card/panel child** ของ layout เดิม แล้วรักษาเงื่อนไขนี้ไว้:

- parent cell ต้องมี `min-width: 0; min-height: 0;`
- panel wrapper ต้องมี `position: relative; height: 100%; overflow: hidden;`
- mount node ของ three.js ต้องถูก resize ผ่าน `ResizeObserver`
- ถ้ามี tabs หรือ collapsible panel ให้ call `handleResize()` หลัง panel ถูกเปิด

แนวคิดนี้เข้ากับ dashboard workshop UI มากกว่าใช้ chart library ที่ assume ว่าตัวเองเป็นหน้าเต็ม.

รายละเอียดการทำงานเชิงโปรटकชั้น

แบบจำลองข้อมูลและการสร้างพื้นผิว

โครงหลักที่แนะนำคือเก็บ **control points** สำหรับแก้ไขจริง แล้ว render ด้วยกริดที่ละเอียดกว่า:

- Editable grid: `9 × 8`
- Render grid: `64 × 48`

วิธีนี้ใกล้เคียงพฤติกรรมของ remap editor จริง เพราะผู้ใช้แก้ไขในตารางหลัก แต่หน้าจอเรนเดอร์เป็นพื้นผิวสั่นขึ้นด้วย **bilinear interpolation**. ใน three.js ใช้ `PlaneGeometry` เป็นฐานซึ่งเป็น geometry class สำหรับระนาบ แล้วอัปเดตค่า position/color ผ่าน buffer attributes ของ geometry. ¹¹

สูตรแนะนำ:

- แกน X = RPM (x1000)
- แกน Z = TPS (%)
- แกน Y = ค่า map (ms หรือ normalized command 0-100)
- ค่าจริงฝั่ง ECU/Map table เก็บเป็น `values[row][col]` โดย row = TPS และ col = RPM

แนวทางนี้ทำให้ export/import JSON ตรงไปตรงมา และต่อกับ backend ภายหลังได้ง่าย

อัลกอริทึมไล่สีแบบ thermal neon

ถ้าต้องการให้เหมือนภาพอ้างอิง ควรไม่ใช่ gradient ธรรมดา 2 สี แต่ใช้ **multi-stop thermal ramp**:

- 0.00 → neon blue
- 0.28 → purple
- 0.58 → magenta
- 0.85 → red
- 1.00 → yellow-white highlight

ในเชิง implementation ให้ normalize ค่า `value / 100` แล้วทำ piecewise interpolation ระหว่าง stop colors ที่ละช่วง วิธีนี้ทำให้ lowlands เป็นสีน้ำเงิน, mid-slope เป็นม่วง/ชมพู, และ peak เป็นแดงสว่างแบบ reference ได้ดีกว่า linear gradient ตรง ๆ

ตัวอย่าง pseudocode:

```
function thermalColor01(t){
  const stops = [
    { t: 0.00, c: '#0d3cff' },
    { t: 0.28, c: '#6b2bff' },
    { t: 0.58, c: '#ff00d4' },
    { t: 0.85, c: '#ff203f' },
    { t: 1.00, c: '#ffd37c' }
  ];
  // find segment and lerp color
}
```

แสง วัสดุ และ bloom ให้ได้อารมณ์ cyber-tech

หากต้องการ “น็อนเรือง” จริงใน three.js ให้คิดเป็น 3 ชั้น:

1. **surface material** ที่มี vertex colors
2. เส้น **wireframe/lattice** สีชมพู/ม่วงโปร่งแสง
3. **post-processing bloom** เพื่อให้จุดสว่างฟุ้ง

เอกสารทางการของ three.js ระบุว่า `Renderer` มี `outputColorSpace` และ `toneMappingExposure`, และ `UnrealBloomPass` ต้องใช้ร่วมกับ tone mapping. ดังนั้น baseline ที่แนะนำคือ:

- `renderer.outputColorSpace = THREE.SRGBColorSpace`
- `renderer.toneMapping = THREE.ACESFilmicToneMapping`
- `renderer.toneMappingExposure ≈ 1.0-1.2`
- `MeshStandardMaterial` หรือ `MeshPhongMaterial` + `vertexColors`
- จุดไฟหลักน้อยดวงแต่คุมทิศชัด เช่น hemisphere + directional + 2 or 3 colored points
- bloom moderate ไม่แรงเกินไป มิฉะนั้น wireframe จะฟุ้งจนอ่านรูปทรงยาก. ¹²

เทคนิค wireframe overlay

ถ้าต้องการลुकใกล้ภาพอ้างอิง ให้หลีกเลี่ยง `WireframeGeometry` อย่างเดียว เพราะมันมักโชว์ diagonal triangle edges เพิ่มมา ซึ่งบางครั้งดู “เป็นตาข่ายสามเหลี่ยม” มากเกินไป. เอกสารของ `WireframeGeometry` ยืนยันว่าใช้สร้าง wireframe จาก `BufferGeometry` ได้สะดวกมาก จึงเหมาะสำหรับ debug หรือโหมดเสริม; แต่ถ้าต้องการลुकแบบ rectangular lattice ให้สร้าง **row lines + column lines** แยกเองจาก dense grid จะควบคุมสไตล์ได้ดีกว่า. ¹³

แนวทางที่ใช้ในตัวอย่างคือ:

- surface mesh = geometry หลัก
- wire rows = line strip ตามแต่ละ row
- wire cols = line strip ตามแต่ละ col
- ยกเส้นขึ้นจากผิวเล็กน้อย เช่น `+0.02` เพื่อกัน z-fighting
- ตั้ง opacity แยก row / col เพื่อให้ภาพมี depth

Performance tips ที่ควรทำก่อนขึ้น production

สิ่งที่ควรทำจริงเมื่อย้ายจาก mockup ไป production:

- จำกัด `devicePixelRatio` เช่น `Math.min(window.devicePixelRatio, 1.5-1.75)` เพื่อลดภาระ fill-rate
- แยก **control grid** ออกจาก **render grid**; อย่า render ละเอียดยกระดับ editor data ตรง ๆ ถ้าต้องการ smooth
- อัปเดต `geometry.computeVertexNormals()` เฉพาะตอน data เปลี่ยน หรือระหว่าง transition สั้น ๆ เท่านั้น
- ใช้ `requestAnimationFrame` เท่าที่จำเป็น; ถ้าไม่มี animation ต่อเนื่องให้ render on-demand
- pause loop ตอน `document.hidden`
- ลดจำนวนไฟ เพราะคู่มือ three.js ระบุชัดว่าแต่ละ light เพิ่มค่า render cost
- ถ้าต้องการหลายระดับคุณภาพ ให้ทำ LOD เช่น `64×48` บน desktop และ `32×24` บน low-power device
- bloom ควรใช้ความแรงพอดีและไม่เรนเดอร์ที่ resolution เกินจำเป็น เพราะแม้ `UnrealBloomPass` จะออกแบบมาให้ quality/performance ดี แต่ก็ยังมีราคาคำนวณของมันเอง. ¹⁴

Accessibility และ fallback

อย่างน้อยควรมี fallback 2 ระดับ:

- 2D heatmap canvas
- sparkline / mini-strip ถ้าพื้นที่แคบมาก

Plotly.js เหมาะเป็น fallback analytics screen ได้เพราะ surface trace มี contour และ lighting controls อยู่แล้ว ส่วน fallback ใน single-file ตัวอย่างนี้ใช้ Canvas 2D เพื่อให้ dependency ต่ำที่สุด. ถ้าต้องรองรับ `prefers-reduced-motion` ให้ปิด damping/transition บางส่วน และลด bloom ลง. ¹⁵

คำสั่งแยกไฟล์และเริ่มใช้งาน

วิธีเริ่มแบบง่ายที่สุด

1. ดาวน์โหลดไฟล์ single-file ด้านบน
2. เปิดด้วย browser ตรง ๆ เพื่อดูผลก่อน
3. ถ้าต้องการทดสอบผ่าน local server:

```
python3 -m http.server 8080
# แล้วเปิด http://localhost:8080/
```

คำสั่งแยก single-file เป็น index.html, styles.css, app.js

สมมติว่าไฟล์ชื่อ `motorboy_3d_surface_map_singlefile.html`

```
mkdir -p motorboy-map-editor
cd motorboy-map-editor
cp ../motorboy_3d_surface_map_singlefile.html ./index.single.html

python3 - <<'PY'
from pathlib import Path
import re

src = Path("index.single.html").read_text(encoding="utf-8")

style_match = re.search(r"<style>(.*?)</style>", src, re.S)
script_match = re.search(r"<script type=\"module\">(.*?)</script>\\s*</body>", src, re.S)

style = style_match.group(1).strip()
script = script_match.group(1).strip()

html = re.sub(
    r"<style>.*?</style>",
    '<link rel="stylesheet" href="./styles.css">',
    src,
    flags=re.S
)

html = re.sub(
    r"<script type=\"module\">.*?</script>",
    '<script type="module" src="./app.js"></script>',
    html,
    flags=re.S
```

```
)

Path("styles.css").write_text(style, encoding="utf-8")
Path("app.js").write_text(script, encoding="utf-8")
Path("index.html").write_text(html, encoding="utf-8")

print("สร้างไฟล์ index.html, styles.css, app.js เรียบร้อย")
PY

python3 -m http.server 8080
```

ถ้าจะใช้ npm/Vite แทน CDN

สร้างโปรเจกต์ใหม่แล้วเอา logic จากไฟล์ single-file ไปวาง:

```
npm create vite@latest motorboy-map-editor -- --template vanilla
cd motorboy-map-editor
npm install three
npm run dev
```

จากนั้นใน `main.js` ใช้ import แบบนี้แทน import map:

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import { EffectComposer } from 'three/addons/postprocessing/EffectComposer.js';
import { RenderPass } from 'three/addons/postprocessing/RenderPass.js';
import { UnrealBloomPass } from 'three/addons/postprocessing/UnrealBloomPass.js';
```

รูปแบบ import ข้างต้นตรงกับเอกสารทางการของ addons. 10

รูปแบบนำเข้าและส่งออกข้อมูล JSON

schema ที่แนะนำ

```
{
  "meta": {
    "app": "MotorBoy ECU Remap Tuner Flash",
    "mapName": "Fuel Map - Injection",
    "version": "1.0.0",
    "units": {
      "x": "RPM x1000",
      "z": "TPS %",
      "y": "ms"
    }
  },
  "rpmAxis": [0, 2, 4, 6, 8, 10, 12, 14, 16],
```



```

"tpsAxis": [0, 15, 30, 45, 60, 75, 90, 100],
"values": [
  [12.2, 14.0, 18.5, 23.1, 27.3, 25.6, 20.2, 17.1, 14.5],
  [16.1, 20.3, 28.8, 35.5, 42.3, 40.1, 33.9, 25.4, 20.2],
  [20.0, 26.5, 38.2, 49.8, 58.1, 57.0, 49.4, 39.1, 30.8],
  [24.2, 31.6, 45.9, 60.2, 71.7, 73.9, 69.1, 58.7, 43.0],
  [28.0, 37.5, 52.4, 68.0, 81.2, 86.5, 82.4, 71.8, 55.1],
  [32.1, 42.4, 56.8, 72.9, 88.6, 94.2, 90.0, 78.3, 60.5],
  [34.4, 44.1, 57.7, 70.0, 82.8, 87.6, 84.4, 72.2, 55.4],
  [35.8, 43.8, 53.0, 61.2, 69.5, 72.0, 68.6, 58.1, 44.0]
]
}

```

หลักการจัดเก็บ

- `rpmAxis.length` ต้องเท่ากับจำนวนคอลัมน์ในทุก row
- `tpsAxis.length` ต้องเท่ากับจำนวนแถวทั้งหมดของ `values`
- `values[row][col]` = ค่าในจุด `(rpmAxis[col], tpsAxis[row])`
- ถ้าภายหลังมี backend ให้ใช้ schema เดียวกันทั้งบนหน้า editor, history snapshot, และ data logger

ในตัวอย่าง single-file ที่แนบไว้ มี helper ให้ใช้งานผ่าน global object:

```

window.MotorBoyMap.exportJSON()
window.MotorBoyMap.importJSON(payload)

```

เช็กลิสต์ส่งต่องานให้ทีมพัฒนา

- ยืนยันว่า **three.js** เป็น **source of truth** สำหรับ remap map panel นี้
- lock เวอร์ชันไลบรารี ไม่ใช้ CDN "latest" ใน production
- จัด parent container ให้ `min-height: 0` และ `overflow: hidden`
- แยก data model ของ map ออกจาก render model
- render grid ใช้ interpolation จาก control grid
- มี 2D fallback และรองรับ reduced-motion
- ทำ export/import JSON ตั้งแต่ต้น เพื่อให้ QA ทดสอบ preset ได้
- วางระบบ state แยกจาก renderer ก่อนต่อกับ backend จริง
- ถ้ามีแผนแปลงเป็น Flutter ภายหลัง ให้ยึด **JSON schema + visual spec** นี้เป็นมาตรฐานกลาง
- อย่าเชื่อมคำสั่ง ECU write/flash จริงใน panel ทดสอบนี้ จนกว่าจะผ่าน safety gate ของระบบทั้งหมด

เอกสารอ้างอิงหลักที่ควรเปิดควบคู่ระหว่างทำงานคือ , , , และ .

1 9 10 <https://threejs.org/docs/pages/OrbitControls.html>
<https://threejs.org/docs/pages/OrbitControls.html>

2 5 15 <https://plotly.com/javascript/3d-surface-plots/>
<https://plotly.com/javascript/3d-surface-plots/>

3 6 <https://doc.babylonjs.com/typedoc/classes/BABYLON.ArcRotateCamera>
<https://doc.babylonjs.com/typedoc/classes/BABYLON.ArcRotateCamera>

4 11 <https://threejs.org/docs/pages/PlaneGeometry.html>

<https://threejs.org/docs/pages/PlaneGeometry.html>

7 <https://deck.gl/>

<https://deck.gl/>

8 <https://app.unpkg.com/three%400.174.0/files/build>

<https://app.unpkg.com/three%400.174.0/files/build>

12 <https://threejs.org/docs/pages/Renderer.html>

<https://threejs.org/docs/pages/Renderer.html>

13 <https://threejs.org/docs/pages/WireframeGeometry.html>

<https://threejs.org/docs/pages/WireframeGeometry.html>

14 <https://threejs.org/manual/en/lights.html>

<https://threejs.org/manual/en/lights.html>